JETBRAINS | Kotlin

Reflection (JVM)

kotlinglang.org/education



What is reflection?

“Reflection can be defined as the ability of a program to manipulate as data something representing the state of the program during its own execution.”*

A simple example of reflection is redefining a class field of an object at runtime.

*Daniel G Bobrow, Richard P Gabriel, and Jon L White. 1993. Object Oriented Programming: The CLOS Perspective (1993), 29–61.

Reflection in Java: an example

Let’s create a simple Kotlin class:

```
class Dog(val name: String, var age: Int) {
   fun bark() = println("bark")
   private fun privateBark() =
       println("private bark!")
   companion object {
       @JvmStatic
       fun publicStaticMethod() =
           println("Hi from public static")
       @JvmStatic
       private fun privateStaticMethod() =
           println("Hi from private static")
       fun publicNotReallyStaticMethod() =
           println("Hi from public not really static")
       private fun privateNotReallyStaticMethod() =
           println("Hi from private not really static")
   }
}
```

Reflection in Java: the main entry point

Instances of the class Class<T> represent classes and interfaces in a running Java (Kotlin) application. **This is the main entry point for reflection.**



Reflection in Java: getting fields

We can get all fields:

```
println("Fields:")
dog::class.java.fields.forEach {
println(it.name) }
```

But the result is an empty list!

```
class Dog(val name: String, var age: Int) {
    fun bark() = println("bark")
    private fun privateBark() =
        println("private bark!")
     ...
}
```

Reflection in Java: getting fields

```
class Dog(val name: String, var age: Int)
```



```
// ================Dog.class ================= //
class version 52.0 (52)
// access flags 0x31
public final class Dog {

// access flags 0x12
private final Ljava/lang/String; name
@Lorg/jetbrains/annotations/NotNull;() // invisible

...
}
```

Kotlin bytecode


```
public final class Dog {
    @NotNull
    private final String name;
    private int age;
    ...
}
```

Java sources

Reflection in Java: getting fields

We can get all **public** fields:

```
println("Fields:")
dog::class.java.fields.forEach {
println(it.name) }
```

We can get **all** fields (with any modifiers):

```
println("Declared fields:")
dog::class.java.declaredFields.forEach {
println(it.name) }
```

Declared fields:

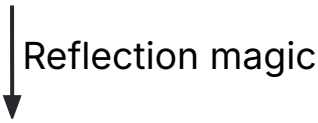
name

age

Reflection in Java: setting fields

We can change **any** field:

```
dog.name = "Bob" // ERROR!!
```

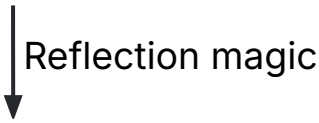



```
dog::class.java.declaredFields.find {
it.name == "name" }?.set(dog, "Bob")
println(dog.name)
```

Reflection in Java: setting fields

We can change **any** field:

```
dog.name = "Bob" // ERROR!!
```




```
dog::class.java.declaredFields.find {
it.name == "name" }?.set(dog, "Bob")
println(dog.name)
```

Exception in thread "main" java.lang.IllegalAccessException: class MainKt cannot access a member of class Dog with modifiers "private final"

at java.base/ jdk.internal.reflect.Reflection.newIllegalAccessException(Reflection.java: 392) at java.base/ java.lang.reflect.AccessibleObject.checkAccess(AccessibleObject.java: 674) at java.base/java.lang.reflect.Field.checkAccess(Field.java:1102) at java.base/java.lang.reflect.Field.set(Field.java:797) at MainKt.main(Main.kt:11)

Reflection in Java: setting fields

We can change **any** field:

```
dog.name = "Bob" // ERROR!!
```

Reflection magic

```
            it.isAccessible = true
            it.set(dog, "Bob")
        }
        println(dog.name)
```

```
dog::class.java.declaredFields.find {
it.name == "name" }?.let{
```

Bob

Reflection in Java: getting class methods

We can get all of the methods in the class:

```
println("Methods:")
dog::class.java.methods.forEach {
println(it.name) }
```

```
class Dog(val name: String, var age: Int) {
    fun bark() = println("bark")
    private fun privateBark() = println("private bark!")

    companion object {
        @JvmStatic
        fun publicStaticMethod() = println("Hi from public static")
        @JvmStatic
        private fun privateStaticMethod() = println("Hi from private static")
        fun publicNotReallyStaticMethod() = println("Hi from public not really static")
        private fun privateNotReallyStaticMethod() = println("Hi from private not really static")
    }
}
```

Methods:

getName

getAge

setAge

bark

publicStaticMethod

wait

wait

wait

equals

toString

hashCode

getClass

notify

notifyAll

Reflection in Java: getting class methods

We can get all of the methods in the class:

```
println("Methods:")
dog::class.java.methods.forEach {
println(it.name) }
```

Standard `Object` methods

Methods:
getName
getAge
setAge
bark
publicStaticMethod
wait
wait
wait
equals
toString
hashCode
getClass
notify
notifyAll

Reflection in Java: getting class methods

We can get all of the methods in our class:

```
println("Methods:")
dog::class.java.methods.forEach {
println(it.name) }
```

Standard thread synchronization methods

Methods:
getName
getAge
setAge
bark
publicStaticMethod
wait
wait
wait
equals
toString
hashCode
getClass
notify
notifyAll

Reflection in Java: getting class methods

We can get all of the methods in our class:

```
println("Methods:")
dog::class.java.methods.forEach {
println(it.name) }
```

```
class Dog(val name: String, var age: Int)
```

For `val` we have only a getter, for `var` we can have both - a getter and a setter

Methods:
getName
getAge
setAge
bark
publicStaticMethod
wait
wait
wait
equals
toString
hashCode
getClass
notify
notifyAll

Reflection in Java: getting class methods

We can get all of the methods in our class:

```
println("Methods:")
dog::class.java.methods.forEach {
println(it.name) }
```

```
class Dog(val name: String, var age: Int) {
    fun bark() = println("bark")
    private fun privateBark() = println("private bark!")

    companion object {
        @JvmStatic
        fun publicStaticMethod() = println("Hi from public static")
        @JvmStatic
        private fun privateStaticMethod() = println("Hi from private static")
        fun publicNotReallyStaticMethod() = println("Hi from public not really static")
        private fun privateNotReallyStaticMethod() = println("Hi from private not really static")
    }
}
```

Methods:
getName
getAge
setAge
bark
publicStaticMethod
wait
wait
wait
equals
toString
hashCode
getClass
notify
notifyAll

Only public methods

No methods from `Companion` (except `JvmStatic`)

Reflection in Java: getting class methods

With `declaredMethods`, we can get all methods (with any modifier), with the exception of inherited methods:

```
println("Declared methods:")
dog::class.java.declaredMethods.forEach { println(it.name) }
```

```
class Dog(val name: String, var age: Int) {
   fun bark() = println("bark")
   private fun privateBark() = println("private bark!")

   companion object {
       @JvmStatic
       fun publicStaticMethod() = println("Hi from public static")
       @JvmStatic
       private fun privateStaticMethod() = println("Hi from
private static")
       fun publicNotReallyStaticMethod() =
println("Hi from public not really static")
       private fun privateNotReallyStaticMethod() = println("Hi
from private not really static")
   }
}
```

Declared methods:

getName

getAge

setAge

bark

privateBark

publicStaticMethod

privateStaticMethod

Reflection in Java: getting class methods

With `declaredMethods`, we can get all methods (with any modifier), with the exception of inherited methods:

```
println("Declared methods:")
dog::class.java.declaredMethods.forEach { println(it.name) }
```

```
class Dog(val name: String, var age: Int) {
   fun bark() = println("bark")
   private fun privateBark() = println("private bark!")

   companion object {
       @JvmStatic
       fun publicStaticMethod() = println("Hi from public static")
       @JvmStatic
       private fun privateStaticMethod() = println("Hi from
private static")
       fun publicNotReallyStaticMethod() =
println("Hi from public not really static")
       private fun privateNotReallyStaticMethod() = println("Hi
from private not really static")
   }
}
```

Declared methods:
getName
getAge
setAge
bark
privateBark
publicStaticMethod
privateStaticMethod

From `Companion` returns only methods with `JvmStatic`

Reflection in Java: getting class methods

```
println("Methods from companion:")
Dog.Companion::class.java.methods.forEach
{ println(it.name) }
```

```
class Dog(val name: String, var age: Int) {
    fun bark() = println("bark")
    private fun privateBark() = println("private bark!")

    companion object {
        @JvmStatic
        fun publicStaticMethod() = println("Hi from public static")
        @JvmStatic
        private fun privateStaticMethod() = println("Hi from private static")
        fun publicNotReallyStaticMethod() = println("Hi from public not really static")
        private fun privateNotReallyStaticMethod() = println("Hi from private not really static")
    }
}
```

Methods from companion:
publicNotReallyStaticMethod
publicStaticMethod
access$privateStaticMethod
wait
wait
wait
equals
toString
hashCode
getClass
notify
notifyAll

Reflection in Java: getting class methods

```
println("Declared methods from
companion:")
Dog.Companion::class.java.declaredMethods.forE
{ println(it.name) }
```

```
class Dog(val name: String, var age: Int) {
    fun bark() = println("bark")
    private fun privateBark() = println("private bark!")

    companion object {
        @JvmStatic
        fun publicStaticMethod() = println("Hi from public static")
        @JvmStatic
        private fun privateStaticMethod() = println("Hi from private static")
        fun publicNotReallyStaticMethod() = println("Hi from public not really static")
        private fun privateNotReallyStaticMethod() = println("Hi from private not really static")
    }
}
```

Declared methods from companion:
publicNotReallyStaticMethod
privateNotReallyStaticMethod
publicStaticMethod
privateStaticMethod
access$privateStaticMethod

Reflection in Java: getting class methods

publicStaticMethod

public final class Dog { public final class Dog$Companion { ... ...

// access flags 0x19 // access flags 0x11

public final static publicStaticMethod()V public final publicStaticMethod()V @Lkotlin/jvm/JvmStatic;() @Lkotlin/jvm/JvmStatic;() L0 Invokes the L0

GETSTATIC Dog.Companion : LDog$Companion; real oneLINENUMBER 10 L0

INVOKEVIRTUAL Dog$Companion.publicStaticMethod ()V ➡LDC "Hi from public static" RETURN ASTORE 1

L1 L1

MAXSTACK = 1 GETSTATIC java/lang/System.out : Ljava/io/PrintStream; MAXLOCALS = 0 ALOAD 1

... INVOKEVIRTUAL java/io/PrintStream.println (Ljava/lang/Object;)V L2 }L3

LINENUMBER 10 L3

RETURN

L4

LOCALVARIABLE this LDog$Companion; L0 L4 0 MAXSTACK = 2

MAXLOCALS = 2

...

}

Reflection in Java: getting class methods

access$privateStaticMethod

```
public final class Dog$Companion {
...
 // access flags 0x12
private final privateStaticMethod()V
@Lkotlin/jvm/JvmStatic;()
 L0
 LINENUMBER 14 L0
 LDC "Hi from private static"
 ASTORE 1
 L1
 GETSTATIC java/lang/System.out : Ljava/io/PrintStream;
 ALOAD 1
 INVOKEVIRTUAL java/io/PrintStream.println (Ljava/lang/Object;)V
 L2
 L3
 LINENUMBER 14 L3
 RETURN
 L4
 LOCALVARIABLE this LDog$Companion; L0 L4 0
 MAXSTACK = 2
 MAXLOCALS = 2
...
}
```

Has a reference from the `Dog` class →

```
public final class Dog {
...
 // access flags 0x1A
 private final static privateStaticMethod()V
 @Lkotlin/jvm/JvmStatic;()
 L0
 GETSTATIC Dog.Companion : LDog$Companion;
 INVOKESTATIC Dog$Companion.access$privateStaticMethod
(LDog$Companion;)V
 RETURN
 L1
 MAXSTACK = 1
 MAXLOCALS = 0
...
}
```

Reflection in Java: getting class methods

access$privateStaticMethod

```
public final class Dog {
...
 // access flags 0x1A
 private final static privateStaticMethod()V
 @Lkotlin/jvm/JvmStatic;()
 L0
 GETSTATIC Dog.Companion : LDog$Companion;
 INVOKESTATIC Dog$Companion.access$privateStaticMethod
(LDog$Companion;)V
 RETURN
 L1
 MAXSTACK = 1
 MAXLOCALS = 0
...
}
```

Cannot call the private

method directly

Reflection from Java: invoking methods

We can invoke methods:

```
dog::class.java.methods.find { it.name ==

"bark" }?.invoke(dog)
```

bark



Pass an object to
invoke the method

Reflection in Java: invoking methods

We can even invoke private methods:

```
dog::class.java.declaredMethods.find {
it.name == "privateBark" }?.invoke(dog)
```

But we need to be careful, or we may get an error like this:

Exception in thread "main" java.lang.IllegalAccessException: class MainKt cannot access a member of class Dog with modifiers "private final"

at java.base/ jdk.internal.reflect.Reflection.newIllegalAccessException(Reflection.java: 392) at java.base/ java.lang.reflect.AccessibleObject.checkAccess(AccessibleObject.java: 674) at java.base/java.lang.reflect.Method.invoke(Method.java:560)

at MainKt.main(Main.kt:26)

Reflection in Java: invoking methods

We can even invoke private methods:

```
dog::class.java.declaredMethods.find { it.name == "privateBark" }?.invoke(dog)
```

```
dog::class.java.declaredMethods.find { it.name == "privateBark" }?.let{
    it.isAccessible = true
    it.invoke(dog)
}
```

private bark!

Reflection in Java: invoking static methods

We can invoke static methods:

```
dog::class.java.declaredMethods.find { it.name == "privateStaticMethod" }?.let{
    it.isAccessible = true
    it.invoke(null)
}
```

Hi from private static



Pass `null` to invoke the method

Reflection from Java: other functions

Other great functions are available in the API!

https://docs.oracle.com/javase/8/docs/technotes/guides/reflection/index.html

There are also lots of other helpful libraries that use the Java Reflection API under the hood, like Reflections and ClassGraph.

Reflection in Kotlin

You can use the Java Reflection API in Kotlin code, but there are some specific language features that it does not cover:

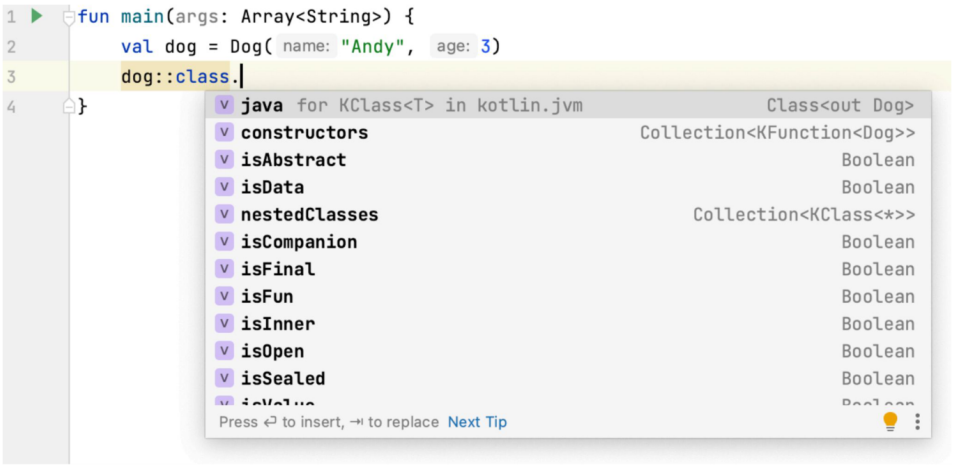
- Data classes
- Nullability
- Top-level functions
- Other Kotlin-specific features

Kotlin has its own API for reflection, which is implemented on top of Java reflection (for JVM).

https://kotlinlang.org/docs/reflection.html

Reflection in Kotlin: an example

Instances of the class KClass<T> represent classes and interfaces in a running Kotlin application. **The main entry point for reflection.**



Reflection in Kotlin: an example

Methods are typically used to check any properties and get member functions, among other things:

```
printIn(dog::class.isAbstract)   false
printIn(dog::class.isFinal)      true
```

```
dog::class.memberProperties.forEach{
println(it.name) }
dog::class.memberFunctions.forEach{
println(it.name) }
```

Don’t forget to include `kotlin-reflect` in the dependencies list!

age

name

bark

privateBark

equals

hashCode

toString

Reflection in Kotlin: usage with types

Reflection can be used with both functions and types:

```
fun myTopLevelFun() = println("My top-
level function")

printIn(::myTopLevelFun.returnType)
printIn(::myTopLevelFun.returnType.isMarkedNul
```

kotlin.Unit

false

Don’t forget to include `kotlin-reflect` in the dependencies list!

Reflection in Kotlin: combining Java and Kotlin entities

You can get KFunction or KProperty by Method or Field from Java via
kotlinFunction or kotlinProperty :

```
println(dog::class.java.returnType)
    .find { it.name == "privateStaticMethod" }?.kotlinFunction)
```

fun Dog.Companion.privateStaticMethod(): kotlin.Unit

Don’t forget to include `kotlin-reflect` in the dependencies list!

Reflection in Kotlin: under the hood

The Kotlin compiler will write a long annotation (a protobuf message) with all the necessary information:

```
@Lkotlin/Metadata;(mv={1, 8, 0}, k=1, d1={"\u0000
\n\u0002\u0018\u0002\n\u0002\u0010\u0000\n\u0000\n\u0002\u0010\u
\u0008\u0008\n\u0002\u0010\u0002\n\u0002\u0008\u0003\u0018\u0000
\u00102\u00020\u0001:\u0001\u0010B\u0015\u0012\u0006\u0010\u0002
\u0004\u001a\u00020\u0005\u00a2\u0006\u0002\u0010\u0006J\u0006\u
\u000f\u001a\u00020\u000eH\u0002R\u001a\u0010\u0004\u001a\u00020
e\n\u0000\u001a\u0004\u0008\u0007\u0010\u0008\"\u0004\u0008\u000
020\u0003\u00a2\u0006\u0008\n\u0000\u001a\u0004\u0008\u000b\u001
d2={"LDog;", "", "name", "", "age", "", "(Ljava/lang/
String;I)V", "getAge", "()I", "setAge", "(I)V",
"getName", "()Ljava/
lang/String;", "bark", "", "privateBark", "Companion",
"reflection"})
```

https://github.com/JetBrains/kotlin/blob/master/core/metadata/src/org/jetbrains/kotlin/metadata/ProtoBuf.java

Why is reflection necessary?

- It’s just really cool!
  - You can manipulate your program however you want.
- It’s usually is used in frameworks, e.g. Spring, JUnit, etc.
  - You can create entities based on the user’s information.
- It can be used to test your application (have access to private fields, methods, etc.).

What are the drawbacks?

- There’s a risk of breaking things in your program.
    - For example, if you use refactoring to rename a method, it will not be renamed automatically in reflection calls, since the reflection may refer to the old name as a string. This can lead to serious problems.
- Your performance will be degraded, as all operations are performed at runtime.
- It does not allow for compile-time optimizations.

Avoid using reflection if you don't really need it!

MethodHandles

-

`MethodHandles` is an API that provides access to various parts of your program:

  - `MethodHandles.Lookup` provides access to fields and methods.
  - `MethodType` defines a method signature with types of parameters and the return type.
  - Methods and fields are accessed with “direct” method handles.
  - Handle adapters are available.

MethodHandles

- `MethodHandles` are not a form of reflection, but they are very similar.
- They can be used for tasks similar to those performed with reflection.

- They work better because JIT optimizations can be applied and fewer call correctness checks are necessary.

MethodHandles

- `MethodHandles.publicLookup()`:

Provides access only to public fields and methods that can be accessed from anywhere.

- `MethodHandles.lookup()`:

Provides access to fields and methods available in the place from which this lookup is called.

MethodHandles: example

-

We can find a getter and invoke it:

```
val lookup = MethodHandles.lookup()
val getProp = lookup.findVirtual(
   Dog::class.java,
   "getName",

MethodType.methodType(String::class.java)
)
println(getProp.invoke(dog))
```

Andy

MethodHandles: example

-

Private methods can be invoked with help from the Reflection API:

```
val privateBarkMethod =
Dog::class.java.getDeclaredMethod("privateBark
privateBarkMethod.isAccessible = true

val privateBarkMH =
lookup.unreflect(privateBarkMethod)
privateBarkMH.invoke(dog)
```

private bark!

MethodHandles: example

-

We can also call any known method:

```
val mt =
MethodType.methodType(Int::class.javaPrimitive
val mh =
lookup.findVirtual(List::class.java,
"size", mt)
val i = mh.invokeExact(listOf(1, 2, 3))
as Int
println(i)
```

```
3
```

`MethodHandles`: example

- We can also call any known method:

```
val mt = MethodType.methodType(Int::class.javaPrimitiveType)
val mh = lookup.findVirtual(List::class.java, "size", mt)
val i = mh.invokeExact(listOf(1, 2, 3)) as Int
println(i)
```

In fact, `invokeExact` provide neither casting to the expected return class nor any conversions (like boxing/unboxing).

MethodHandles: example

- We can also call any known method:

```
val mt = MethodType.methodType(Int::class.javaPrimitiveType)
val mh = lookup.findVirtual(List::class.java, "size", mt)
val i = mh.invokeExact(arrayOf(1, 2, 3)) as Int
println(i)
```

```
Exception in thread "main" java.lang.invoke.WrongMethodTypeException: expected (List)int but found (Integer[])int
        at java.base/java.lang.invoke.Invokers.newWrongMethodTypeException(Invokers.java:523)
        at java.base/java.lang.invoke.Invokers.checkExactType(Invokers.java:532)
        at MainKt.main(Main.kt:71)
```

Takeaways

- If you have the opportunity to solve a problem without reflection, do it.
- If you can't solve a problem without reflection, consider alternatives like MethodHandler.
- The Java Reflection API can be awkward to use, consider using Kotlin reflection instead.
- Be prepared to catch very strange and unusual bugs! :)

JETBRAINS | Kotlin

Thank you

